P1864R0
Defining Target Tuplets

Published Proposal, 2020-07-08

This version:
https://wg21.link/P1864R0

Latest published version:
https://wg21.link/P1864

Author:
Isabella Muerte

Audience:
SG15

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest source:
src/target-tuplets.bs

This source:
src/target-tuplets.bs

Table of Contents

§ 1. Revision History

§ 1.1. Revision 0

Initial release. 🎉

§ 2. Motivation

One of the more difficult things in the vast breadth of build systems is declaring what system architecture, vendor, operating system, and environment is used when compiling for a given platform. In some cases, these names are chosen at random by various implementations and differ from each vendor. While some time ago these would have been called "Target Triples", in practice they have been "Target Tuplets".

This paper aims to define and describe the concept of a target tuplet and how they can be consumed by build systems, toolchains, and users alike. This paper *does not* attempt to define or limit existing target tuplets and instead defers this "definition list" to a separate Standing Document. The reason for this is that updating target tuplets with a Technical Report or Standard requires several years to be updated and voted in. This model can’t react to changes in existing practices, while a Standing Document can be updated at each plenary vote, or simply be updated by an editor, and published at will.

This paper might give examples of existing target tuplets, but this is for demonstration purposes and is not to be taken as a guarantee that the tuplet will exist in such a form in the future.

Lastly, this paper also does not attempt to define a *sysroot*. That may or may not be defined by a future separate paper.

§ 2.1. Why Tuplet instead of Triplet or Tuple?

Traditionally, a target tuplet has been called a *triplet*, as this referred to the *architecture*, the *operating system*, and the *environment*. However, as time has gone on, a 4th field was added, while the name *triplet* was kept. While the author is not a verified expert on the English language, *triple* and the *number 4* do not mean the same thing.

Additionally, C++ has the concept of a `std::tuple`. The author opted to select the term *tuplet* to remove any possibly ambiguity regarding discussions of the definition found within this proposal.

Lastly, the `-et` and `-ette` suffixes are used to describe something small (e.g., a cas*ette* is a small case). However, `-et` also means a group or grouping, such as in the definition of oct*et*. A target tuplet is a grouping of several elements regarding a target, hence the use of the term *tuplet* and not *tuplette*.

§ 3. Design Considerations

A target tuplet is written (in utf-8) as a series of words (text), delimited via a "hyphen-minus" (-). The current layout of a target tuplet is borrowed from clang, as other compilers tend to support this layout but also provide alternative layouts that are incompatible with clang or each other.

Typically, a target tuplet *does not* infer floating point ABI, processor specific intrinsics, or FPU specific intrinsics. These are more granular and defining them at a broad scope is, in the authors opinion, fruitless.

Lastly, a target tuplet can be considered "ill-formed, no diagnostic required" if an incorrect sequence is put together and passed to the compiler. For example, specifying `mips-ibm-cuda-macho` as a target would most likely not work in any existing compiler (at the time of this writing). Instead, it is up to users and build systems to ensure that these incorrect sequences do not make their way to the compiler. This differs from clang's approach, which is to convert any unrecognized entries as `unknown`, and then infer specifics.

§ 4. Wording

§ 4.1. Terms and Definitions

For the purposes of this document, the following terms apply

build system
: A program that will drive the execution of a C++ compiler on a set of translation units. They live a level above the compiler, and thus are technically out of scope for the C++ standard document itself.

platform
: The inverse of the C++ abstract machine. That is, a concrete set of compilers, tools, programs, and code necessary to execute a program. Colloquially referred to as a system in some spaces.

vendor
: An entity that provides an implementation of a C++ toolchain.

host
: A specific instance of a platform, upon which the C++ compiler executes. [*Note*: The C++ standard does not mandate that a C++ compiler must be a well-formed program itself —*end note*]

target
: A specific instance of a platform, upon which a well-formed program is planned to execute

machine
: A specific instance of a target, upon which a well-formed program can run.

object file format
: The detailed file format a translation unit is in prior to linking.

executable file format
: The detailed file format a program is in once linked.

§ 4.2. Target Tuplet

§ 4.2.1. In general

[1] Target tuplets are a series of components used to represent a human friendly phrase for a compiler target. They are useful for discussing minimum feature sets for a code base, as well as what platform a program is expected to compile and execute on.To be able to create a well-formed program for a given target, a build system must know the details of how to invoke a compiler, invoke a linker, and produce a final program or collection of object files.

[2] Since build systems live outside of the standard, they must be able to invoke one or more compilers, regardless of whether they are from the same vendor. This document defines 4 parts that make up a target tuplet.

[3] The 4 parts that make up a target tuplet are

(3.1) — architecture

(3.2) — vendor

(3.2) — operating system

(3.2) — environment

respectively. The generic term *component* refers to any portion of a target tuplet that meets the above criteria.

[4] Target tuplets are constructed by concatenating the content of all components with the hypen-minus character (-). Components are not permitted to contain the hyphen-minus themselves.

[5] A target tuplet is considered well-formed if a compiler is able to understand the 4 components provided. Likewise, if a compiler is unable to understand the 4 components provided, the tuplet is considered ill-formed. While a build system is free to resolve the validity of a target tuplet, only a compiler can verify whether or not it is well formed.

[6] In addition to the requirements above, a compiler or build system is free to take fewer than the 4 components required of a target tuplet. Such a sequence is called a *supplied target tuplet*. Each missing component in a supplied target tuplet can be replaced with the catch-all name `unknown`, however at least one component must be supplied by name. [*Note:* In most cases, the `vendor` component will be ignored or missing —*end note*]

§ 4.3. Component Descriptions

§ 4.3.1. Architecture

[1] The architecture refers to the instruction set used for a given processor.

[2] The architecture may not refer to specific releases of names of a given processor.

[3] The architecture is permitted to express revisions to the instruction set itself.

[4] [*Note:* `arm`, `armv6`, and `armv7a` are from the same family of processors, however the `v6` and `v7` refer to revisions made to the ISA —*end note*]

§ 4.3.2. Vendor

[1] A vendor is any entity that provides a C++ toolchain to a user. Whether this is an organization, a simple person, a PC, or even no one. [2] In the event that no one is marked as the vendor, the component will always resolve to the value `none`.

§ 4.3.3. Operating System

[1] An *operating system* is a superset of an environment, but is not responsible of how a user interacts with their toolchain.

§ 4.3.4. Environment

[1] An *environment* is any one of an executable file format, an ABI or a runtime.

[2] Due to the nature of how an environment can be expressed, each type of environment can supercede another, with the exception of an executable file format.

(2.1) An executable file format implies that there is only 1 way to load and execute the program without virtualizing or converting to another environment.

(2.2) An ABI implies the existence of an executable file format, an object file format, and 1 or more calling conventions for symbols within the object file format

(2.3) A runtime is an object file format, an ABI for object files, and 1 or more libraries needed for a an executable to be produced

§ 4.3.5. Resolution

[1] Resolving a supplied target tuplet into a well-formed target tuplet is not an intuitive process based on the written form.

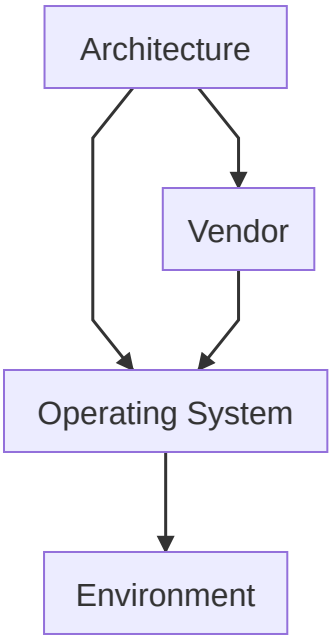

Figure 1 *Written Component Order*

[2] While the written component order is currently `architecture`, `vendor`, `operating system`, and `environment`, the actual priority differs based on the importance of each component.

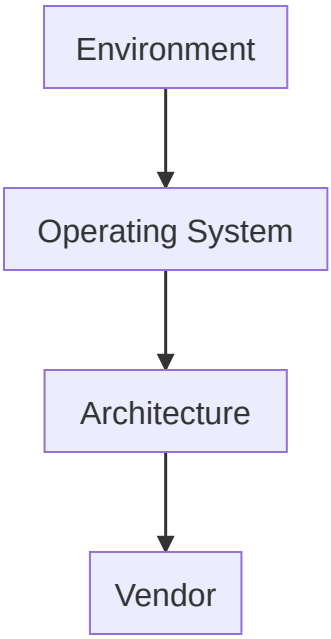

Figure 2 *Component Priority Order*

[3] In the event that no operating system was provided, the component will be set to the *bare metal specifier* `none` and the resolution steps are to start again.

§ 5. Informative Explanation

The road to get to the point where we can have a definition of a target tuplet has been a long one. There are many platforms, processors, toolchains, external tools, build systems, and configurations to consider and many of these are older than Standard C++ by almost a decade (The initial release of C++ in 1985 was during the beginning of very early stages of this "wild west" of cross compilation). This section is intended to explain some of the decisions made with wording, as well as some of the history of how we ended up at the point we are currently at. Additionally, a lot of work was done to formalize some of the wording and to "

§ 5.1. A Brief History of Compiler Targets

Many older compilers are no longer available. Their source code, binaries, documentation, etc. are either lost to time, or require vast sums of money to gain access to. As such, the author only had GCC, a few non-triplet configured C compilers, and a handful of usenet postings to go off of for the information located here.

> Note: This portion of the document is not meant to be a "Howard Zinn-esque" retelling of how target tuplets came to be.

Before autotools, GCC's early releases used simple shell scripts that were written by hand. The changelog was only a small file with a few notes, and some light information on how to configure the build for porting to new platforms. These files were labelled `config-<processor>`, as the ability to target a specific toolchain wasn't really a possibility at the time. As time went on, the number of targets that GCC could run on or target grew. This is when the first set of target triplets were created. At the time, the second field of a triplet was what is now the *third entry* in a target tuplet. At some point, an optional *vendor* field was created. In the wild, some might be familiar with the "triplet" of `x86_64-none-linux-gnu`, which is also sometimes represented as `x86_64-unknown-linux-gnu` or `x86_64-pc-linux-gnu`. A list of wildcard host/targets is provided by GCC.

Because of the success of GCC, this form of targeting a platform became the *de facto* way of defining a target. GCC also added several aliases over the years (e.g., permitting `*-*-*-linux` to mean `*-*-*-elf` on some platforms), which led to a bit of confusion on how to target a platform. Once Clang supported targets, they did not provide aliases, which reduced the surface area for confusion.

§ 5.2. Why Vendor Over Machine

At the Denver 2019-09 SG15 meeting, the author was recommended via a poll result to investigate naming conventions for the second entry as to whether it should be *vendor* or *machine*. After some time, it became clear that the *vendor* entry has always been for vendors. The use of `*-pc-*` was taken instead to mean "IBM Compatible PC", back when that used to mean something.

However, when users have taken to think the second entry is a machine, operating system, or platform, the compiler has effectively *normalized* the second field to either be `unknown` or something else. Thus, using *machine* does not work well, as the wording above states that *vendor* is optional in a *supplied target tuplet*.

§ 5.3. Why environment over X

Some other names came up during discussion of the fourth field. These included, `runtime`, `ABI`, etc. In actuality, this last field has historically been where everything is thrown into one place to differentiate the compiler's target from effectively everything else. In practice, however, there is a hierarchy implied by *what* is in this field.

If an object file format, such as `PE`, `COFF`, `a.out`, `Mach-O`, or `ELF` is given, it typically implies that there are 0 or 1 default vendor specific ABI or runtime available. In other words, these *could* be freestanding compiler targets, or there is only one possible runtime provided by the compiler. e.g., in the case of `*-apple-*-macho`, there is only ONE ABI and runtime available, and it is provided by Apple, with the object file format being `Mach-O`. In the case of a `*-apple-*-coff`, there is still only ONE ABI and runtime available, but the object file format is now different.

A level above this, is an ABI. e.g., `gnu`, or `msvc`. These imply all of:

- an object file format (e.g., ELF and PE/COFF)
- 1 or more calling conventions for symbols within the object file format
- multiple runtimes that can co-exist on a given platform.

This is why, for the uninformed, MinGW and MSVC cannot link against each other unless it is done via C and unless the "symbols" agree upon calling convention.

Above this, we have *runtimes*. These have all of:

- A specific object file format
- An ABI for storing precompiled object file formats

- 1 or more libraries needed for a program to load without issue.

As an example, `mingw` generates object files under the PE/COFF format, supports both the `__stdcall` and `__cdecl` calling conventions, and supplies a runtime that allows GCC compiled code to run directly on Windows. While the actual format of the PE/COFF files generated by MinGW are not compatible with MSVC, this doesn’t change that MinGW has its own runtime, even if said runtime is simply to reorganize symbols so that it can safely link against the MSVC runtime.

Likewise, `cygwin` generates object files under the PE/COFF format, but targets a POSIX-like environment. Thus it requires a library for its generated programs to execute correctly.

Because of this tiered system, an "umbrella" term is needed. Our toolchains do not exist in a vacuum, and interact with our overall *environment*, hence the name *environment* was chosen.

§ References

§ Informative References

[CLANG-CROSS-COMPILE]
Cross-compilation using Clang. URL: https://clang.llvm.org/docs/CrossCompilation.html

[GCC-HOST-TARGET]
Host/Target specific installation notes for GCC. URL: https://gcc.gnu.org/install/specific.html